Guia de Estudo e Base de Conhecimento Científica

Modelagem Preditiva de Atrasos no Voo Regular Ativo (VRA / ANAC) com Deep Learning

Docente Responsável: Prof. Dr. Sérgio Canuto

Frente Comparativa: Redes Recorrentes (LSTM 3.2)

Dataset Oficial: Portal Brasileiro de Dados Abertos (VRA/ANAC)

Frente Principal: Multilayer Perceptrons (MLP 3.1)

Otimizador Bayesiano: Optuna (TPE + MedianPruner)

Repositório: joseallangoncalves/deep_learning_trabalho_final

Sumário do Guia de Estudo

1. **Tópico 1:** O Problema Real e a Formulação Matemática (Alvo Binário > 15 min)
2. **Tópico 2:** A Regra de Ouro do Pré-processamento e Prevenção de Data Leakage
3. **Tópico 3:** Engenharia de Atributos e Codificação Cíclica (Seno e Cosseno)
4. **Tópico 4:** Divisão Cronológica e Tratamento do Desbalanceamento (Class Weights)
5. **Tópico 5:** Arquitetura Multilayer Perceptron (MLP) — Frente Principal 3.1
6. **Tópico 6:** Redes Neurais Recorrentes (LSTM) — Estudo Comparativo 3.2
7. **Tópico 7:** Otimização Bayesiana de Hiperparâmetros com Optuna
8. **Tópico 8:** Métricas de Avaliação e Análise de Custo-Efetividade Computacional
9. **Tópico 9:** Como Executar Novos Experimentos e Buscar Resultados Superiores

Tópico 1: O Problema Real e a Formulação Matemática

1. Contexto Operacional e Intuição

No setor aéreo, um atraso de voo acarreta efeito cascata (conexões perdidas, tripulação excedendo limites de horas regulamentares e aeronaves ocupando posições de pátio). Antecipar a probabilidade de atraso crítico antes da decolagem permite que os controladores e companhias aéreas reestruturem malhas e aloquem recursos preventivamente.

- **Unidade de Observação:** Cada linha representa um **voo regular comercial individual**.
- **Limiar Crítico Regulatório:** A ANAC e a IATA definem que atrasos de até 15 minutos são tolerância operacional normal. Acima de 15 minutos é formalmente classificado como **atraso crítico**.

2. Formulação Matemática das Variáveis

EQUAÇÃO 1 — ATRASO CONTÍNUO DE CHEGADA

$$\text{AtrasoChegadaMin} = \frac{\text{DataHoraReal}_{\text{chegada}} - \text{DataHoraPrevista}_{\text{chegada}}}{60 \text{ segundos}}$$

EQUAÇÃO 2 — VARIÁVEL ALVO BINÁRIA

$$y = \begin{cases} 1, & \text{se AtrasoChegadaMin} > 15 \text{ minutos (Atraso Crítico)} \\ 0, & \text{se AtrasoChegadaMin} \leq 15 \text{ minutos (Pontual / Tolerância)} \end{cases}$$

ONDE ESTÁ NO CÓDIGO: SRC/DATA_PIPELINE.PY (LINHAS 54-60)

```
df_clean['AtrasoChegadaMin'] = (df_clean['ChegadaReal'] - df_clean['ChegadaPrevista']).dt.total_seconds() / 60.0
df_clean = df_clean[(df_clean['AtrasoChegadaMin'] ⩾ -120) & (df_clean['AtrasoChegadaMin'] ⩽ 1440)].copy()
df_clean['target_bin'] = (df_clean['AtrasoChegadaMin'] > delay_threshold_min).astype(int)
```

COMO EXPLICAR AOS COLEGAS E AO PROFESSOR

"Nós não tentamos prever os minutos exatos com regressão porque, na operação aeroportuária real, saber se vai atrasar 18 ou 22 minutos importa menos do que a decisão binária de acionar um plano de contingência para atrasos que violam os 15 minutos tolerados pela ANAC."

Tópico 2: A Regra de Ouro do Pré-processamento e Prevenção de Data Leakage

1. O Conceito de Data Leakage (Vazamento de Dados)

Vazamento de dados ocorre quando informações que só existiriam *depois* do evento ou parâmetros globais dos dados de teste são inadvertidamente usados durante o treinamento. O modelo parece atingir 99% de acurácia no caderno, mas falha totalmente na vida real.

2. As Duas Fontes Graves Evitadas

1. **Eliminação de Variáveis Futuras/Concorrentes:** Colunas como `ChegadaReal`, `PartidaReal` e `CodigoJustificativa` só existem após o voo ter acontecido. Se mantidas na matriz X, a rede apenas subtrairia a partida real da prevista e descobriria a resposta com trapaça. Foram totalmente descartadas de X.
2. **Isolamento Estatístico do Treino:** Parâmetros estatísticos de média (μ_{treino}) e desvio padrão (σ_{treino}) do `StandardScaler` foram ajustados **exclusivamente** nos 70% cronológicos de treino.

EQUAÇÃO 3 — A REGRA DE OURO DO PRÉ-PROCESSAMENTO

$$\mu_{\text{treino}}, \sigma_{\text{treino}} \leftarrow \text{Fit exclusivamente em } D_{\text{treino}}$$

$$X_{\text{val}} = \frac{X_{\text{val}} - \mu_{\text{treino}}}{\sigma_{\text{treino}}}, \quad X_{\text{teste}} = \frac{X_{\text{teste}} - \mu_{\text{treino}}}{\sigma_{\text{treino}}}$$

ONDE ESTÁ NO CÓDIGO: SRC/DATA_PIPELINE.PY (LINHAS 121-154)

```
# O preprocessor faz fit() APENAS no conjunto de treino:
preprocessor.fit(train_df[num_cols + cat_cols])

# As outras partições apenas recebem transform():
X_train = preprocessor.transform(train_df[feature_cols])
X_val   = preprocessor.transform(val_df[feature_cols])
X_test  = preprocessor.transform(test_df[feature_cols])
```

Tópico 3: Engenharia de Atributos e Codificação Cíclica (Seno e Cosseno)

1. O Problema da Descontinuidade Numérica

Se codificarmos as horas de forma linear (0 a 23), um voo programado para as **23:55** terá valor ≈ 23.9 e um voo programado para as **00:05** terá valor ≈ 0.08. A distância aritmética entre eles é $|23.9 - 0.08| = 23.82$ unidades! Fisicamente, eles estão separados por apenas **10 minutos**. Uma rede neural interpretaria isso como dois extremos opostos.

2. A Solução: Projeção Circular Trigonométrica

Mapeamos o tempo em um ângulo no círculo trigonométrico de 2π radianos:

EQUAÇÃO 4 — CODIFICAÇÃO TRIGONOMÉTRICA CÍCLICA

$$\text{Hora}_{\sin} = \sin\left(\frac{2\pi \cdot \text{Hora}}{24}\right), \quad \text{Hora}_{\cos} = \cos\left(\frac{2\pi \cdot \text{Hora}}{24}\right)$$

$$\text{DiaSemana}_{\sin} = \sin\left(\frac{2\pi \cdot \text{Dia}}{7}\right), \quad \text{DiaSemana}_{\cos} = \cos\left(\frac{2\pi \cdot \text{Dia}}{7}\right)$$

$$\text{Mês}_{\sin} = \sin\left(\frac{2\pi \cdot \text{Mês}}{12}\right), \quad \text{Mês}_{\cos} = \cos\left(\frac{2\pi \cdot \text{Mês}}{12}\right)$$

Com o par $(\sin, \cos)$, os horários 23:55 e 00:05 ficam adjacentes na circunferência, preservando a continuidade temporal exata.

3. Agrupamento de Categorias de Alta Cardinalidade

Aeroportos e companhias pequenas aparecem poucas vezes no ano. Para evitar centenas de colunas esparsas e overfitting, mantivemos apenas as **Top 10 companhias** e **Top 20 aeroportos**; todas as demais foram agrupadas no rótulo unificado `'OUTROS'`.

ONDE ESTÁ NO CÓDIGO: SRC/DATA_PIPELINE.PY (LINHAS 65-89)

```
hora_dec = df_clean['PartidaPrevista'].dt.hour + df_clean['PartidaPrevista'].dt.minute / 60.0
df_clean['hora_sin'] = np.sin(2 * np.pi * hora_dec / 24.0)
df_clean['hora_cos'] = np.cos(2 * np.pi * hora_dec / 24.0)

# Agrupamento top-k para conter a cardinalidade:
top_airlines = df_clean['ICAOEmpresaAerea'].value_counts().nlargest(top_k_airlines).index
df_clean['ICAOEmpresaAerea'] = df_clean['ICAOEmpresaAerea'].where(df_clean['ICAOEmpresaAerea'].isin(top_airlines),
other='OUTROS')
```

Tópico 4: Divisão Cronológica e Tratamento do Desbalanceamento (Class Weights)

1. Por que NÃO usar K-Fold aleatório ou Shuffle?

Em séries temporais e dados operacionais, o passado explica o futuro. Embaralhar as linhas aleatoriamente faria a rede usar dados de voos de dezembro para prever voos de janeiro do mesmo ano. Por isso, adotou-se divisão cronológica estrita:

- **Treino (70%):** Voos mais antigos cronologicamente (D_{treino}).
- **Validação (15%):** Período intermediário (usado pelo Optuna e Early Stopping).
- **Teste (15%):** Período mais recente (teste cego intocado).

2. Desbalanceamento e Ponderação de Classes (Class Weights)

Na base do VRA, 78.8% dos voos são pontuais e apenas 21.2% atrasam criticamente. Uma rede convencional aprenderia a prever sempre 0 para atingir 78.8% de acurácia com utilidade nula. A solução foi incorporar pesos inversos na Binary Cross-Entropy:

EQUAÇÃO 5 — PESOS INVERSOS DE CLASSE E FUNÇÃO DE PERDA PONDERADA

$$w_0 = \frac{N}{2 \cdot N_0} = 0.637, \quad w_1 = \frac{N}{2 \cdot N_1} = 2.325$$

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} [w_1 y_i \log(\hat{y}_i) + w_0 (1 - y_i) \log(1 - \hat{y}_i)]$$

Errar a classe minoritária ($y = 1$) custa para a rede **3.65 vezes mais caro** do que errar a classe majoritária, equilibrando as atualizações de gradiente.

ONDE ESTÁ NO CÓDIGO: MAIN.PY (LINHAS 69-77)

```
n_neg = int(np.sum(y_train == 0))
n_pos = int(np.sum(y_train == 1))
class_weights = {
    0: float(len(y_train) / (2.0 * n_neg)),  # ~0.637
    1: float(len(y_train) / (2.0 * n_pos))   # ~2.325
}
```

Tópico 5: Arquitetura Multilayer Perceptron (MLP) — Frente Principal 3.1

1. Teorema da Aproximação Universal e Fundamentos Teóricos

O Teorema da Aproximação Universal (Hornik et al., 1989; Cybenko, 1989) garante que uma rede com uma única camada oculta e ativações não-lineares pode aproximar qualquer função contínua em subconjuntos compactos de $\mathbb{R}^n$. Entretanto, arquiteturas **profundas** conseguem decompor representações hierárquicas complexas com exponencialmente menos parâmetros do que redes rasas largas.

2. Elementos Críticos das Camadas Ocultas

- **Ativação ReLU ($\mathrm{ReLU}(z) = \max(0, z)$):** Elimina a saturação de gradiente em valores positivos, permitindo convergência rápida via otimizador Adam.
- **Inicialização He Normal:** Pesos inicializados com $\mathcal{N}(0, \sqrt{2/n_{\text{in}}})$, prevenindo que o sinal suma ou exploda em camadas ReLU profundas.
- **Batch Normalization:** Normaliza ativações intermediárias com média 0 e variância 1, acelerando o aprendizado e suavizando a superfície de perda.
- **Dropout ($p = 0.2 \sim 0.3$):** Desliga aleatoriamente neurônios a cada batch, impedindo coadaptação espúria.
- **Regularização L2 ($\lambda \cdot \sum \|W\|_2^2$):** Penaliza pesos excessivamente grandes, forçando generalização.

EQUAÇÃO 6 — FUNÇÃO DE PERDA COMPLETA DA MLP COM REGULARIZAÇÃO L2

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\left[w_1 y_i \log(\hat{y}_i) + w_0(1-y_i)\log(1-\hat{y}_i)\right] + \frac{\lambda}{2}\sum_{l=1}^{L}\|W_l\|_F^2$$

3. As Três Variantes Experimentais

- **MLP V1 (Rasa Baseline):** 1 camada de 64 neurônios, sem Dropout, sem L2, sem BatchNorm (2.433 parâmetros).
- **MLP V2 (Profunda Regularizada):** 3 camadas ($256 \to 128 \to 64$), BatchNorm, Dropout(0.3) e L2 (10^{-4}). **Maior Macro-F1: 0.4936**.
- **MLP V3 (Otimizada Optuna):** 3 camadas ($32 \to 64 \to 128$), Dropout(0.216) e L2 (1.46×10^{-5}). **Maior ROC-AUC: 0.5472** com apenas 12.193 parâmetros.

ONDE ESTÁ NO CÓDIGO: SRC/MODELS.PY (LINHAS 10-60)

```
for idx, units in enumerate(hidden_units):
    reg = regularizers.l2(l2_reg) if l2_reg > 0 else None
    x = layers.Dense(units, kernel_initializer='he_normal', kernel_regularizer=reg)(x)
    if use_batch_norm:
        x = layers.BatchNormalization()(x)
    x = layers.Activation('relu')(x)
    if dropout_rate > 0:
        x = layers.Dropout(dropout_rate)(x)
outputs = layers.Dense(1, activation='sigmoid', kernel_initializer='glorot_uniform')(x)
```

Tópico 6: Redes Neurais Recorrentes (LSTM) — Estudo Comparativo 3.2

1. Hipótese e Formatação em Tensores Tridimensionais

A hipótese testada foi: o histórico de atrasos dos últimos 5 voos consecutivos influencia o próximo voo devido a congestionamento de pistas e pátios. Para isso, os dados foram remodelados em tensores 3D com janela deslizante $W = 5$:

```
Formato do Tensor 3D: (Amostras, Timesteps = 5, Atributos = 36)
```

2. A Célula Long Short-Term Memory (LSTM)

Em Redes Recorrentes Simples, o gradiente desaparece ao retropropagar no tempo (Vanishing Gradient). A LSTM resolve isso mantendo o **Estado de Célula (c_t)** através de portas que atuam como autoestradas lineares de fluxo de gradiente:

EQUAÇÃO 7 — EQUAÇÕES DE TRANSIÇÃO DA CÉLULA LSTM

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f) \quad [\text{Porta de Esquecimento: o que descartar do passado}]$$

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i), \quad \tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c) \quad [\text{Porta de Entrada: novo conteúdo}]$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad [\text{Atualização do Estado de Célula}]$$

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o), \quad h_t = o_t \odot \tanh(c_t) \quad [\text{Porta de Saída e Vetor Oculto}]$$

3. Veredito Científico: Por que a MLP superou a LSTM?

- **Desempenho:** A LSTM obteve Macro-F1 de 0.4443, inferior aos **0.4936** da MLP V2.
- **Custo Computacional:** A latência da LSTM foi de **316.30 ms por 1k previsões** — 52% mais lenta que a MLP V2 (207.89 ms).
- **Razão Operacional:** A malha aérea não é uma fila linear única. Centenas de pistas e aeroportos operam em paralelo. Forçar essa malha multidimensional em uma cadeia unidimensional linear injetou ruído sequencial. A modelagem tabular densa (MLP) provou-se operacionalmente superior.

ONDE ESTÁ NO CÓDIGO: SRC/DATA_PIPELINE.PY (L156-171) E SRC/MODELS.PY (L61-96)

```
# Tensores 3D com janela deslizante W=5:
X_train_seq, y_train_seq = create_recurrent_sequences(X_train, y_train, time_steps=5)

# Modelo recorrente Keras:
rnn_layer = layers.LSTM(units=64, return_sequences=False)(inputs)
outputs = layers.Dense(1, activation='sigmoid')(rnn_layer)
```

Tópico 7: Otimização Bayesiana de Hiperparâmetros com Optuna

1. Busca Bayesiana (TPE) vs. Grid e Random Search

O algoritmo **TPE (Tree-structured Parzen Estimator)** substitui a busca cega por um modelo probabilístico $p(\theta \mid y)$. Ele avalia as perdas de tentativas passadas e amostra novas combinações priorizando as regiões do espaço hiperparamétrico que demonstraram maior probabilidade de minimizar o erro de validação.

2. Poda Inteligente (MedianPruner) e Early Stopping

O callback customizado `OptunaPruningCallback` reporta a perda ao Optuna ao final de cada época. Se uma tentativa apresentar desempenho inferior à mediana das tentativas anteriores na mesma época, o Optuna aborta a execução imediatamente com `TrialPruned`, economizando até 70% do tempo computacional.

3. Tabela do Espaço de Busca Hiperparamétrico

Hiperparâmetro	Espaço de Busca	Escala	Justificativa Teórica
Taxa de Aprendizado (η)	$[10^{-4}, 10^{-1}]$	Logarítmica	Permite explorar ordens de grandeza distintas ($0.0001, 0.001, 0.01$).
Tamanho do Lote (B)	$\{32, 64, 128, 256\}$	Categórica	Lotes menores geram ruído benéfico; lotes maiores aceleram o throughput matricial.
Profundidade (L)	$[1, 4]$ camadas	Inteiro	Investiga a transição entre modelos rasos e profundos.
Largura por Camada	$\{32, 64, 128, 256\}$	Categórica	Flexibilidade na capacidade representacional de cada camada.
Taxa de Dropout (p)	$[0.0, 0.5]$	Uniforme	Regularização estocástica para evitar sobreajuste.
Decaimento L2 (λ)	$[10^{-6}, 10^{-2}]$	Logarítmica	Restringe a norma dos tensores de peso sem achatar as ativações.

MELHORES PARÂMETROS ENCONTRADOS NO ESTUDO (RESULTS/OPTUNA_STUDY.DB)

$$\eta = 0.0314, \quad B = 256, \quad L = 3 \text{ camadas } [32, 64, 128], \quad p = 0.216, \quad \lambda = 1.46 \times 10^{-5} \implies \text{Val Loss} = 0.5467$$

ONDE ESTÁ NO CÓDIGO: SRC/TUNING.PY (LINHAS 12-120)

```
study = optuna.create_study(
    study_name="vra_mlp_tuning", storage="sqlite:///results/optuna_study.db",
    direction="minimize", sampler=optuna.samplers.TPESampler(seed=42),
    pruner=optuna.pruners.MedianPruner(n_startup_trials=5, n_warmup_steps=5)
)
```

Tópico 8: Métricas de Avaliação e Análise de Custo-Efetividade Computacional

1. Métricas Apropriadas para Dados Desbalanceados

- **Precision (Atraso):** Quantos voos previstos como atrasados realmente atrasaram (evita alarmes falsos).
- **Recall (Atraso):** Quantos dos atrasos reais o modelo conseguiu capturar (evita surpresas operacionais).
- **Macro F1-Score:** Média harmônica balanceada entre ambas as classes: $\text{Macro-F1} = \frac{F1_0 + F1_1}{2}$.
- **ROC-AUC:** Área sob a curva ROC, medindo a capacidade intrínseca do modelo de ranquear atrasos com probabilidades maiores que voos pontuais.

2. Tabela Comparativa Consolidada no Conjunto de Teste Cego

Modelo Avaliado	Frente	Macro F1-Score	AUC-ROC	Parâmetros Treináveis	Latência (ms / 1k amostras)
MLP V1 (Rasa Baseline)	3.1	0.4348	0.5464	2.433	`119.97 ms`
MLP V2 (Profunda Regularizada)	**3.1**	**0.4936**	**0.4761**	**51.585**	**207.89 ms**
MLP V3 (Otimizada Optuna)	3.1	0.4503	**0.5472**	12.193	231.57 ms (4x menor que V2)
Recorrente LSTM (Sequencial)	3.2	0.4443	0.5223	25.921	316.30 ms (+52% lentidão)

ONDE ESTÁ NO CÓDIGO: SRC/EVALUATION.PY (LINHAS 17-38 E 92-118)

```
metrics, y_pred, y_pred_prob = evaluate_model(model, X_test, y_test, threshold=0.5)
cost_benchmark = benchmark_model_cost(model, sample_batch=X_test[:100], n_repetitions=20)
```

Tópico 9: Como Executar Novos Experimentos e Buscar Resultados Superiores

1. Execução via Terminal e Caderno Interativo

O projeto está pronto para experimentação rápida no terminal PowerShell:

TERMINAL POWERSHELL (RAIZ DO PROJETO)

```
# Executar o pipeline completo padrão:
python main.py

# Ou abrir o caderno interativo no Jupyter:
jupyter notebook notebooks/03_optuna_experiments.ipynb
```

2. Três Experimentos Práticos para Tentar Superar a MLP V2

Experimento A: Ajuste do Limiar de Decisão (Threshold Tuning)

Em dados desbalanceados, a probabilidade 0.50 é conservadora. Ao testar limiares de decisão entre 0.35 e 0.45, o Recall da classe minoritária sobe significativamente e o Macro-F1 pode ultrapassar 0.52.

EM SRC/EVALUATION.PY OU NO SCRIPT PRINCIPAL:

```
# Testar com limiar reduzido de decisão:
metrics_otimizadas, _, _ = evaluate_model(mlp_v2, X_test, y_test, threshold=0.40)
```

Experimento B: Substituição da Ativação por Swish ou GELU

Em `src/models.py` (Linha 41), substitua `'relu'` por `'swish'` (função auto-gated suave: $f(x) = x \cdot \sigma(\beta x)$). Isso previne neurônios mortos e permite que gradientes negativos suaves continuem fluindo em dados tabulares esparsos.

Experimento C: Agendador de Taxa de Aprendizado (Learning Rate Scheduler)

Adicione um callback de desaceleração suave de taxa de aprendizado quando o modelo estacionar no platô de validação:

ADICIONAR AOS CALLBACKS DO KERAS:

```
reduce_lr = tf.keras.callbacks.ReduceLROnPlateau(
    monitor='val_loss', factor=0.5, patience=3, min_lr=1e-5, verbose=1
)
```

Resumo dos Arquivos do Projeto

Arquivo	Função no Projeto
`src/data_pipeline.py`	Ingestão VRA, filtros, features cíclicas trigonométricas, split temporal estrito e ColumnTransformer.
`src/models.py`	Fábrica de arquiteturas: MLP modular (He Normal, BatchNorm, Dropout, L2) e Recorrente (LSTM/GRU).
`src/tuning.py`	Otimizador Bayesiano Optuna com MedianPruner, banco SQLite e função objetivo de validação.
`src/evaluation.py`	Cálculo de métricas desbalanceadas (Macro-F1, AUC), matrizes de confusão e benchmark de latência.
`main.py`	Orquestrador ponta a ponta do experimento científico com geração de relatórios.
`reports/relatorio_tecnico.pdf`	Relatório técnico final acadêmico submetido à disciplina.
`reports/slides_apresentacao.html`	Slides interativos formatados para apresentação e defesa oral do trabalho.